



ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

Lecture 9: Approximation Algorithms – Part 2

Review:

- An approximation algorithm returns a solution to a combinatorial optimization problem that is **provably** close to optimal (as opposed to a *heuristic* that may or may not find a good solution).
- Approximation algorithms are typically used when finding an optimal solution is intractable.
- But they can also be used in some situations where a near-optimal solution can be found quickly and an exact solution is not needed.

Estimating the chromatic number of G , i.e. $\chi(G)$:

$$G = (V, E), |V| = n$$

$$\frac{n}{\alpha(G)} \leq \chi(G) \leq n - \alpha(G) + 1$$

Question: Is this estimation useful to us in terms of determining an approximation? Although this estimation limits the interval, it does not really help us for an acceptable approximation. In the following we consider special examples where this interval is arbitrarily filled.

Complete bipartite Graph $K_{n,n}$:

$$\alpha(K_{n,n}) = n$$

$$\chi(K_{n,n}) = 2$$

$$\frac{2n}{\alpha(K_{n,n})} \leq \chi(K_{n,n}) \leq 2n - \alpha(K_{n,n}) + 1$$

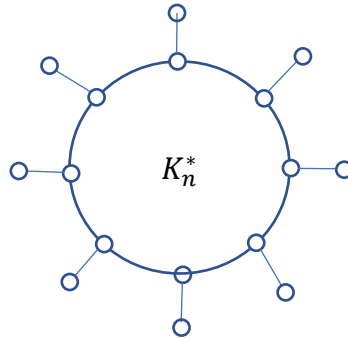
$$\frac{2n}{n} \leq 2 \leq 2n - n + 1$$

$$2 \leq 2 \leq n + 1$$



ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

(Modified) Complete Graph K_n^* :



$$\alpha(K_n^*) = n$$

$$\chi(K_n^*) = n$$

$$\frac{2n}{n} \leq n \leq 2n - n + 1$$

$$2 \leq n \leq n + 1$$

Question: Can the coloring problem be approximated at all?

Theorem: Let A be a polynomial algorithm to approximate $\chi(G)$ of an undirected graph G with $|OPT(G) - A(G)| \leq k$, then it follows that $P = NP$

Assuming the reverse of the theorem, i.e. $P \neq NP$, it follows that there can be no polynomial algorithm for approximating $\chi(G)$.

Instead of approximation algorithms, we must use and analyze them in terms of their goodness.

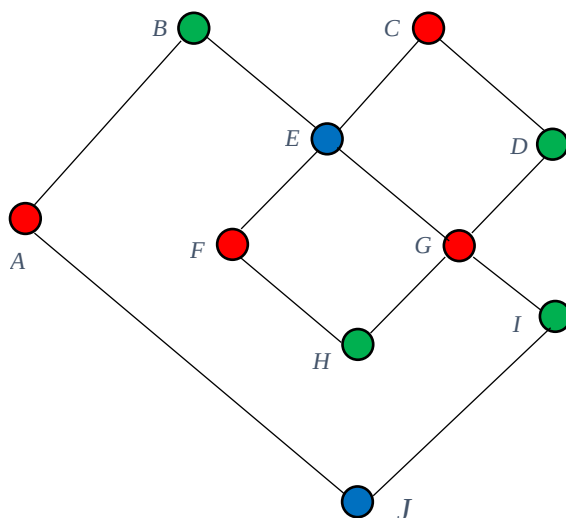
➔ 2 well-known classical heuristic algorithms: Greedy and Johnson Coloring Algorithms



Greedy Coloring Algorithm:

```
algorithm greedyColoring;  
  input  $G = (V, E)$  : Graph;  $V = \{1, 2, \dots, n\}$   
  
  output  $f: [1 \dots \max]$  of int;  
     $color$ : int;  
  
  var  $counter, color, i, j$ : int;  
   $f := 0$ ;  $color := 0$ ;  $counter := 0$ ;  
  
  while  $counter < n$  do  
     $color := color + 1$ ; for all  $i \in V$  do  
      if  $f[i] < 1$  and  $f[i] \neq -color$  then  
         $f[i] := color$ ;  
         $counter := counter + 1$ ;  
        for all  $j \in NG(i)$  do  
          if  $f[j] < 1$  then  
             $f[j] := -color$   
          endif  
        endfor  
      endif  
    endfor  
  endwhile  
  return  $f$ ;  
  return  $color$ ;  
endalgorithm greedyColoring.
```

Colors = { R: Red, G: Green, B: Blue }





ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

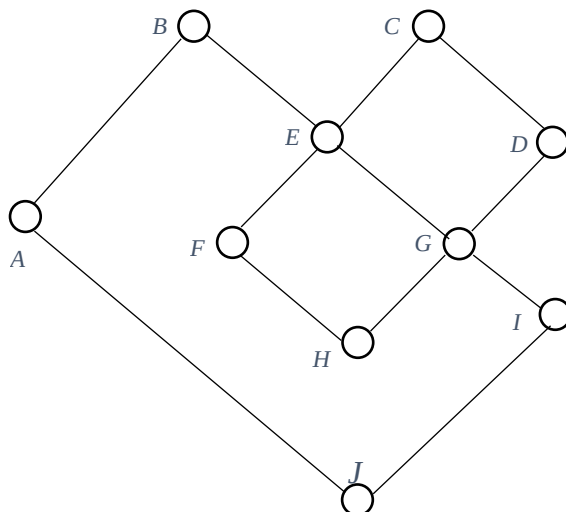
Let $G = (V, E)$ be a graph with $|E| = m$, then it applies

$$\text{greedyColoring}(G). \text{farbe} \leq \lceil \sqrt{2m} \rceil$$

Johnson Coloring Algorithm:

```
algorithm johnsonColoring;  
  input  $G = (V, E)$  : Graph;  
  output  $f: [1 \dots \max]$  of int;  
     $color$  : int;  
  var  $U, W$  : subset of  $V$ ;  
   $f := 0; color := 0; W := V$ ;  
  
  repeat  
     $U := W$ ;  
     $color := color + 1$ ;  
    while  $U \neq \emptyset$  do  
      Determine node  $u$  in  $G_{IS}$  with minimal grade;  
       $f[u] := color$ ;  
       $U := U - \{u\} - NU(u)$ ;    //  $NU(u)$ : Neighbors of  $u$   
       $W := W - \{u\}$ ;  
    endwhile  
  until  $W = \emptyset$ ;  
  return  $f$ ;  
  return  $color$ ;  
endalgorithm johnsonColoring.
```

Colors = { **R**: Red, **G**: Green, **B**: Blue }





ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

Let $G = (V, E)$ be a graph with $|V| = n$, then it applies

$$johnsonColoring(G).farbe \leq \left\lfloor \frac{4n \cdot \log(\chi(G))}{\log n} \right\rfloor$$

Some examples for approximation problems:

(1) Fully approximable:

- a. Triangle Traveling Salesman Problem
- b. 2-Processor-Scheduling

(2) Partially approximable:

- a. Vertex/Node Cover

(3) Not approximable:

- a. Traveling Salesman Problem
- b. Coloring Problem